MLLP EXPERIENTIAL LEARNING GROUP ASSIGNMENT

# Image Caption Generator

Image-to-text description generation using BLIP

Project Report | Topic 16

## Group details

| GROUP MEMBER | ROLL NUMBER | CONTRIBUTION |
|---|---|---|
| Shourya Basu | 2330400-Basu | To be added by the group |
| Manit Kumar Sahu | 2430605-Manit | To be added by the group |
| Ayushman Mahapatra | 2330020-Ayushman | To be added by the group |
| Shrutik Sinha | 2330404-Shrutik | To be added by the group |

Submission date: 10 November 2026

The group supplied the member names and roll numbers. Each member's actual contribution is left for the group to complete.

# Project overview

This project implements a local application that accepts an image and generates a short English description with the pretrained BLIP image-captioning model. The Streamlit interface validates the upload, prepares the image, runs inference, displays the generated caption, and lets the user download it as text. The prototype uses the model as released and does not fine-tune it.

The included sample run shows both useful summaries and a clear model error. BLIP described the cat and bicycle images in broad terms, while it misidentified a dog sculpture at a park entrance as a cat. This illustrates why a generated caption needs human review.

## 1. Problem statement and objectives

Images communicate visual information, but people and applications often need a short text description for access, search, or cataloguing. The project task is to map one image to one natural-language caption using an image-captioning model.

### Objectives

- Accept common image formats through a simple interface.
- Check file type, size, dimensions, and image decoding before inference.
- Convert the image to the format expected by BLIP and generate a caption.
- Display the caption and offer a text download.
- Record representative outputs and describe limitations honestly.

### Expected outcome

A working local prototype that gives a concise description of the main visible subject or scene. The caption is an aid to review, not a guaranteed factual record of every image detail.

## 2. Proposed methodology and processing pipeline

The app keeps the uploaded image in memory. It applies EXIF orientation correction, converts pixels to RGB, and passes the image to the Hugging Face BLIP processor. The processor prepares pixel values for the visual encoder. The decoder generates token IDs with beam search. The app decodes those tokens into a caption and shows the result.

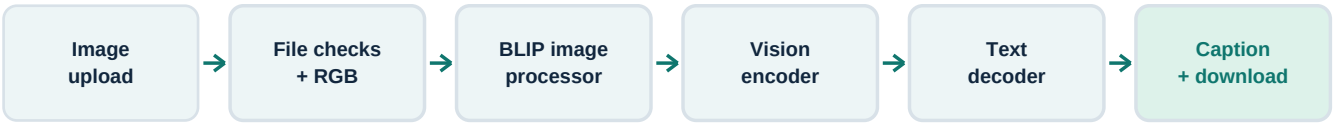



*Figure 1. Processing pipeline from upload to caption output.*

Configuration: `Salesforce/blip-image-captioning-base`, `num_beams=3`, and `max_new_tokens=32`. The model is loaded once and cached by Streamlit. The app uses CUDA when PyTorch detects a CUDA device; this sample evaluation ran on CPU.

# 3. Models and tools used

| Component | Use in the project |
|---|---|
| BLIP base image captioning model | Pretrained vision-language checkpoint generates captions from images. Model ID: Salesforce/blip-image-captioning-base [1, 2]. |
| Hugging Face Transformers | Loads the processor and model from the Hugging Face Hub and runs text generation [3]. |
| PyTorch | Tensor operations, model inference, and CPU/GPU device selection. |
| Streamlit | Browser-based upload interface and cached model resource [4]. |
| Pillow | Image decoding, EXIF orientation, and RGB conversion. |

The model card describes this checkpoint as a BLIP base model for image captioning, pretrained on COCO, and identifies a BSD-3-Clause license. This project performs inference only and does not train on COCO or another dataset [2].

# 4. Implementation and usability

The project is implemented in Python 3.11 with a Streamlit interface. Uploaded files are limited to JPG, PNG, and WebP, 10 MB, and 25 megapixels. Invalid or oversized images are rejected. The app does not write the upload to disk. On first use, Transformers downloads the model from Hugging Face; subsequent runs reuse the local model cache.

## Run instructions

1. Create and activate a Python virtual environment.
2. Install dependencies with `python -m pip install -r requirements.txt`.
3. Start the app with `streamlit run app.py`.
4. Open the local address shown in the terminal, upload an image, and select `Generate caption`.
5. Use `python caption_image.py path/to/image.jpg` for the command-line interface.

## Practical uses

The prototype can assist with draft alt text, photo organization, and searchable image collections. A person should review each caption, especially when accuracy matters.

# 5. Results and sample outputs

The model was run on three CC0 sample images included in the repository. The output below is verbatim from the local CPU run. Inference time includes processor and generation work after model loading; it excludes model loading and the initial download.

| | |
|---|---|
|  | **dog_park.jpg**<br><br>**Caption:** a gate with a cat sitting on top of it<br><br>**Observation:** The model noticed the entrance but called an animal sculpture a cat and missed that it is a dog park. |
|  | **cat_portrait.jpg**<br><br>**Caption:** a small white cat standing in a pile of wood<br><br>**Observation:** The model identified the main subject and described the surrounding wood. |
|  | **bicycle_in_grass.jpg**<br><br>**Caption:** a bike is laying in the middle of a garden<br><br>**Observation:** The model identified the bicycle and greenery but did not mention that the bicycle is partly hidden. |

Evaluation protocol: inspect whether each caption identifies the main subject and scene. This is a qualitative review of three images, not a statistically representative test set. No BLEU, CIDEr, or other benchmark score is claimed.

## Evaluation environment

| Setting | Observed value |
|---|---|
| Python / PyTorch | 3.11.9 / 2.14.0+cpu |
| Transformers / Streamlit | 5.22.2 / 1.67.0 |
| Device | CPU, no CUDA GPU detected |
| Generation times | Dog park 2.222 s; cat 2.367 s; bicycle 2.472 s |

# 6. Limitations

- The checkpoint generates English captions and may miss small or obscured objects.
- It can confuse similar objects or describe a sculpture as a real animal, as the dog-park example shows.
- It can omit relations or add details that are not visible. The model provides no calibrated confidence score in this interface.
- The first run needs an internet connection and about 1 GB of model weights. CPU generation is slower than GPU generation on many computers.
- Three manually reviewed images cannot establish broad accuracy or fairness across scenes and people.

# 7. Conclusion and future scope

The prototype completes the image-to-caption pipeline with a pretrained BLIP model, upload validation, CPU/GPU selection, caption display, and text download. The examples show that the model can summarize a clear subject while still making an object-level error in a complex scene.

Future work could evaluate a larger image set with human reference captions, report standard captioning metrics, add multilingual output, improve review controls, and collect user feedback. These extensions need their own data and evaluation plan.

## Presentation and repository handoff

The editable slides and a 4 to 5 minute recording script are included in the project folder. The group still needs to record the presentation, state each member's actual contribution, and add the video or an accessible video link to the repository.

# 8. References and repository link

[1] J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation," 2022. arXiv:2201.12086.

[2] Salesforce AI Research, BLIP image-captioning model card, accessed 10 November 2026.

[3] Hugging Face, Transformers image-captioning task guide, accessed 10 November 2026.

[4] Streamlit, file uploader and resource caching documentation, accessed 10 November 2026.

[5] Sample images are from Wikimedia Commons. Each contributor released the corresponding image under CC0 1.0. File pages: dog park, cat, and bicycle. Contributor names are listed in the sample-image credits.

GitHub repository URL: To be added after the group publishes the repository.